

الموضوع العاشر | تحديد طول زمن التشغيل القادم لوحدة المعالجة المركزية (CPU Burst)

- تعتمد الخوارزمية على تقدير مسبق لأزمة التشغيل، ويتم ذلك باستخدام المتوسط الأسّي عبر المعادلة التالية:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

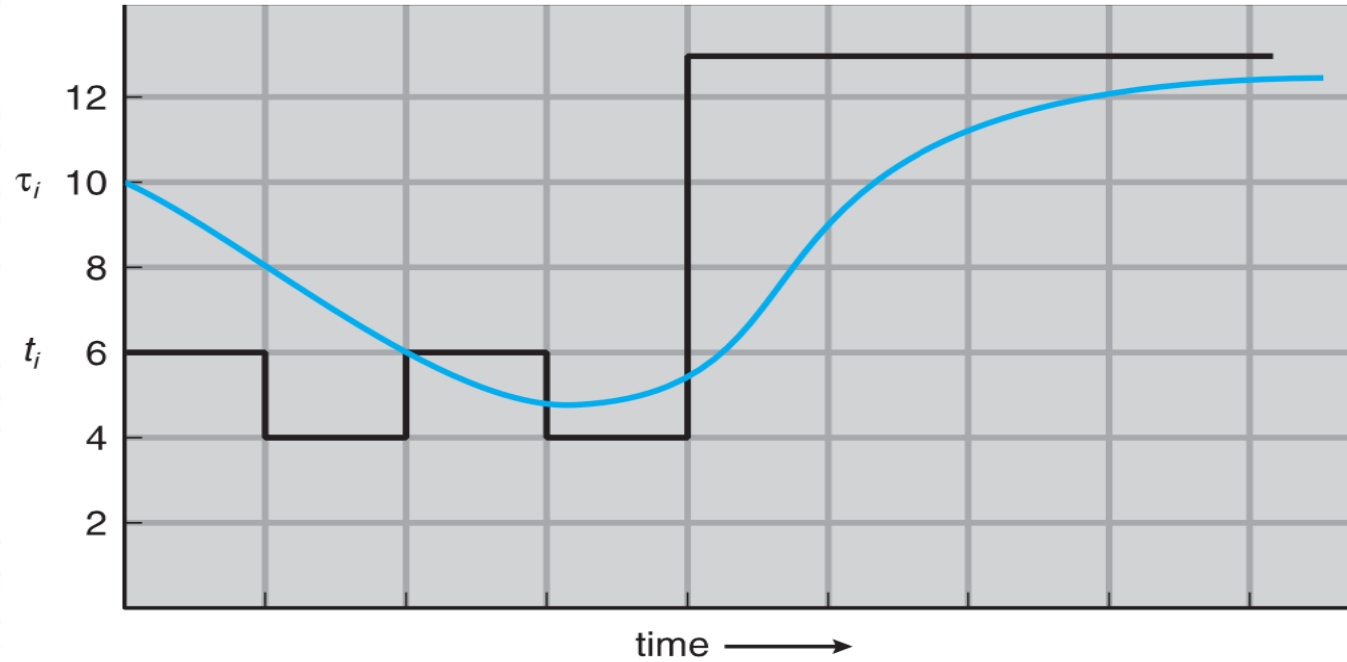
حيث:

- τ_{n+1} هو التقدير المتوقع لزمن التشغيل القادم.
- t_n هو الزمن الفعلي الذي استغرقته العملية أثناء آخر تشغيل لها على المعالج.
- τ_n هو الزمن الذي "يتوقعه" النظام قبل ما تبدأ العملية.
- α هو قيمة تتراوح بين 0 و 1، تُستخدم لتحديد مدى تأثير القيمة الحالية مقارنة بالقيمة السابقة في عملية التقدير. غالباً ما يتم حسابه $= 0.5$.

الموضوع العاشر | تحديد طول زمن التشغيل القادم لوحدة المعالجة المركزية (CPU Burst)

- $\alpha, 0 \leq \alpha \leq 1$
- عندما تكون $\alpha=0$:
 - في هذه الحالة، الزمن المتوقع للحدث التالي τ_{n+1} يعتمد تمامًا على الزمن المتوقع السابق τ_n .
 - يعني، كل ما يحدث هو أننا نتوقع أن الزمن سيكون مشابهًا تمامًا لما حدث في آخر مرة.
- عندما تكون $\alpha=1$:
 - في هذه الحالة، الزمن المتوقع يعتمد كليًا على الزمن الفعلي الذي حدث في الدورة السابقة، أي τ_n .
 - وبذلك، لا يُؤخذ التقدير السابق في الحسبان. فقط آخر زمن تشغيل فعلي (CPU Burst) يتم احتسابه
 - المعادلة يمكن توسيعها على النحو التالي:

الموضوع العاشر | تحديد طول زمن التشغيل القادم لوحدة المعالجة المركزية (CPU Burst)



CPU burst (t_i)	6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	9	11	12	...

الموضوع العاشر | مثال على أقصر وقت متبق أولًا (Shortest-remaining-time-first)

- هنا نضيف مفاهيم اختلاف أوقات الوصول والاستباقية إلى التحليل.

Process	Arrival Time	(Burst Time)
P1	0	8
P2	1	4
P3	2	9
P4	3	5

- مخطط جانت لخوارزمية أقصر زمن تشغيل استباقية (Preemptive SJF)



- متوسط زمن الانتظار = $(10-1)+(1-1)+(17-2)+(5-3)/4 = 26/4 = 6.5$ m sec

الموضوع العاشر | جدولة الأولوية (Priority Scheduling)

- تُمنح كل عملية رقم أولوية (قيمة رقمية)، وتُخصص وحدة المعالجة المركزية للعملية ذات الأولوية الأعلى (أي ذات الرقم الأصغر).
- أنواع جدولة الأولويات:
 - استباقية
 - غير استباقية
 - خوارزمية أقصر زمن تشغيل أولاً (SJF)
- تُعطى الأولوية للعملية التي يُتوقع أن تستغرق أقل زمن في التنفيذ، بناءً على تقدير وقت التشغيل المتوقع
- مشكلة التجويع (Starvation)
قد لا تُنفَّذ العمليات ذات الأولوية المنخفضة إذا وُجدت عمليات ذات أولوية أعلى باستمرار، مما يؤدي إلى بقائها في قائمة الانتظار لفترات طويلة.
- الحل - آلية التقدّم (Aging)
تقنية لزيادة أولوية العمليات التي تنتظر طويلاً، بهدف منع تأخيرها وضمان تنفيذها تدريجياً.

الموضوع العاشر | جدولة الأولوية (Priority Scheduling)

<u>Process</u>	<u>(Burst Time)</u>	<u>Priority</u>
P ₁	10	3
P ₂	1	1
P ₃	2	4
P ₄	1	5
P ₅	5	2



• مخطط جانت لجدولة الأولويات

• متوسط زمن الانتظار = 8.2 ملل بالثانية

تم تنفيذ العمليات بالترتيب: $P_4 \leftarrow P_3 \leftarrow P_1 \leftarrow P_5 \leftarrow P_2$ حسب الأولوية (كلما كان الرقم أصغر، كانت الأولوية أعلى).

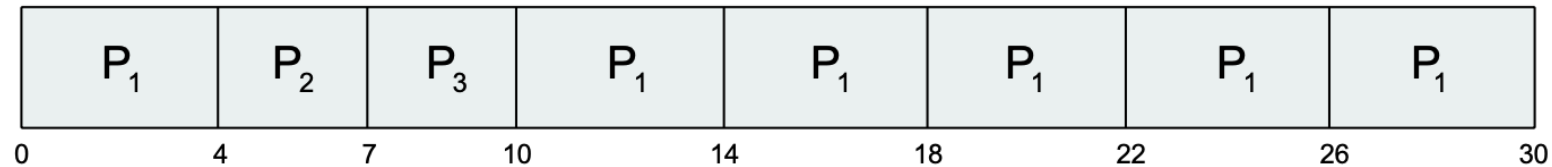
الموضوع العاشر | جدولة التقسيم الزمني (Round Robin – RR)

- تعتمد هذه الجدولة على تخصيص وحدة زمنية صغيرة لكل عملية، تُعرف باسم الوقت الكمي (Quantum)، يتراوح غالباً بين 10 إلى 100 ميلي ثانية.
- بعد انتهاء الوقت الكمي للعملية، يتم إيقافها مؤقتاً وإعادتها إلى نهاية قائمة العمليات الجاهزة، لإعطاء فرصة للعمليات الأخرى.
- إذا كانت هناك n عملية جاهزة قائمة العمليات الجاهزة وكان الوقت الكمي هو q فإن كل عملية تحصل على q/n وحدة زمنية بشكل دوري.
- يتم تقسيم وقت المعالج الى أجزاء من q و لا تنتظر أي عملية أكثر من $q(n-1)$ وحدة زمنية.
- يتم استخدام مقاطعة زمنية في كل مرة ينقضي فيها الوقت الكمي، لتحديد العملية التالية في الدور.
- الأداء:
- إذا كان q كبيراً، تصبح الجدولة مشابهة لسياسة القდوم أولاً يُخدم أولاً FIFO.
- إذا كان q صغيراً، تزداد عدد عمليات التبديل بين العمليات (تبديل السياق)، مما يؤدي إلى حمل زائد على النظام.

الموضوع العاشر | مثال على جدولة التقسيم الزمني (RR) مع الوقت الكمي = 4

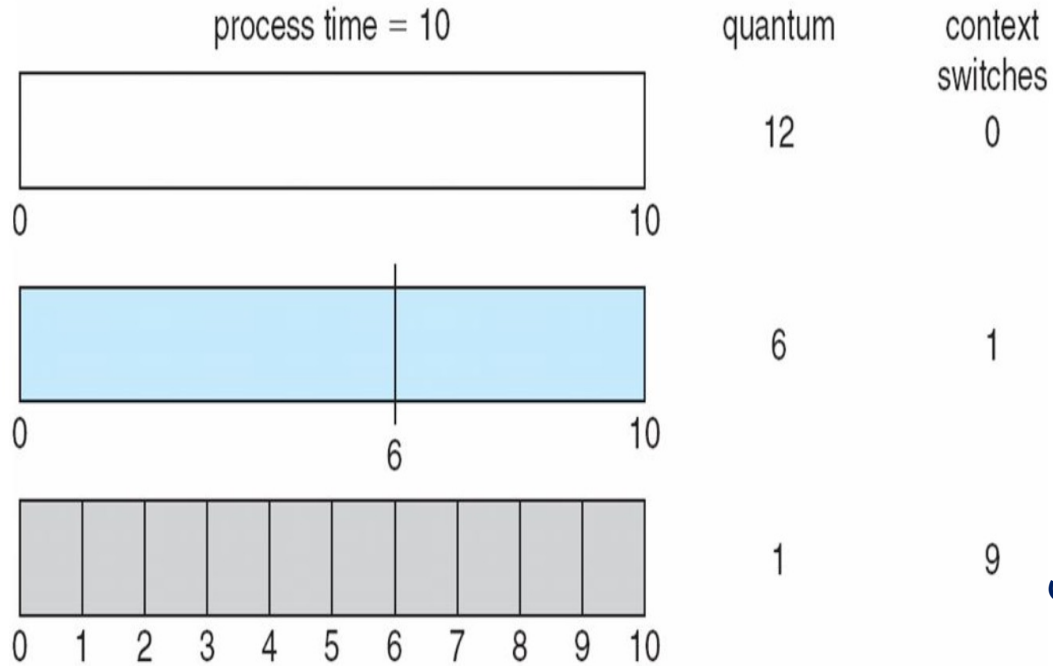
<u>Process</u>	<u>(Burst Time)</u>
P_1	24
P_2	3
P_3	3

- مخطط جانت لجدول التقسيم الزمني :



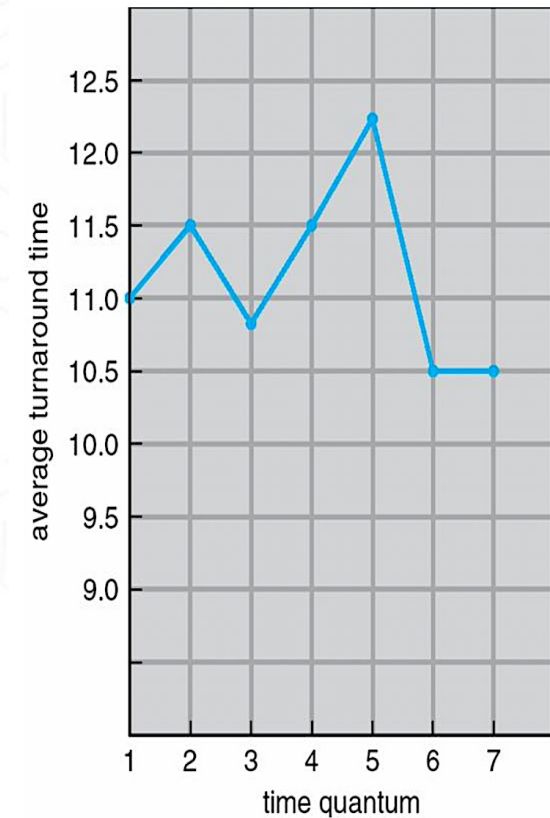
- عادةً، يكون متوسط زمن الإنهاء أعلى مقارنةً بجدولة أقصر زمن تشغيل أولًا (SJF) لكنه يوفر استجابة أفضل.
- يجب أن يكون الوقت الكمي كبيرًا مقارنةً بزمن تبديل السياق:
- عادةً، يكون الوقت الكمي بين 10 ملل بالثانية و 100 ملل بالثانية.
- زمن تبديل السياق أقل من 10 ميكرو ثانية.

الموضوع العاشر | الوقت الكمي وزمن تبديل السياق



- يوضح الشكل تأثير تغيير قيمة الوقت الكمي Quantum على عدد تبديلات السياق Context Switches خلال تنفيذ عملية زمنها 10 وحدات.
- $12 = \text{Quantum}$ لا يتم إيقاف العملية، لذا لا يحدث أي تبديل سياق. عدد تبديلات السياق $= 0$
- $6 = \text{Quantum}$ يتم تنفيذ العملية لمدة 6 وحدات، ثم يتم إيقافها مؤقتًا واستئنافها لاحقًا لإكمال الزمن المتبقي. عدد تبديلات السياق $= 1$
- $1 = \text{Quantum}$ يتم تقطيع العملية إلى 10 أجزاء زمنية صغيرة، مع تبديل سياق في كل مرة. عدد تبديلات السياق $= 9$
- الاستنتاج: كلما قلّ الوقت الكمي، زاد عدد تبديلات السياق، مما يؤدي إلى زيادة الحمل الزائد على النظام نتيجة العمليات الإضافية اللازمة لتبديل السياق.

الموضوع العاشر | زمن الإنهاء يتغير مع الوقت الكمي



process	time
P_1	6
P_2	3
P_3	1
P_4	7

80% of CPU bursts
should be shorter than q

يجب أن تكون 80% من دفعات وحدة
المعالجة المركزية (CPU Bursts)
أقصر من الوقت الكمي q .

يعرض الرسم البياني العلاقة بين الوقت الكمي Time Quantum وزمن الإنهاء المتوسط Average Turnaround Time لمجموعة من العمليات. يتضح من المخطط أن تغيير قيمة الوقت الكمي يؤثر بشكل مباشر على زمن الإنهاء، حيث يمكن أن يؤدي تقليل الوقت الكمي إلى زيادة عدد مرات التبديل بين السياقات Context Switches وبالتالي رفع زمن الإنهاء، في حين أن زيادة الوقت الكمي قد تقلل التبديل، ولكنها تؤثر على استجابة النظام للعمليات القصيرة.

الموضوع العاشر | الجدولة متعددة المستويات (Multilevel Queue)

- يتم تقسيم قائمة الجاهزية (Ready Queue) إلى عدة قوائم مستقلة وفقاً لطبيعة العمليات، ومن الأمثلة عليها:
 - الواجهة الأمامية (Foreground) مخصصة للعمليات التفاعلية
 - الواجهة الخلفية (Background) مخصصة للعمليات الدفعية (Batch).
- تحتفظ كل عملية بمكانها في قائمتها المخصصة طوال فترة تنفيذها، ولا يتم نقلها بين القوائم.
- يتم تخصيص خوارزمية جدولة مستقلة لكل قائمة حسب طبيعة العمليات الموجودة فيها، على النحو التالي:
 - الواجهة الأمامية (Foreground) تُستخدم خوارزمية الجدولة الدائرية (RR).
 - الخلفية (Background) تُستخدم خوارزمية القاجم أولاً يخدم أولاً (FCFS).

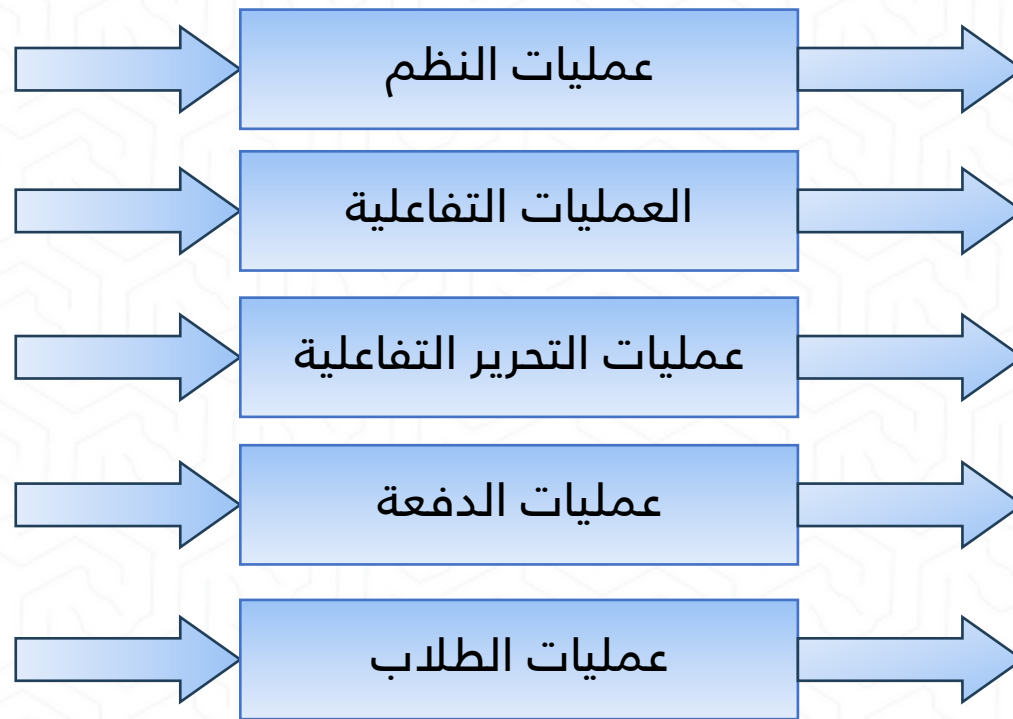
الموضوع العاشر | الجدولة متعددة المستويات (Multilevel Queue)

أساليب الجدولة بين القوائم:

- الجدولة ذات الأولوية الثابتة (Fixed Priority Scheduling)
 - تُنفذ العمليات في القوائم حسب أولوية محددة مسبقًا (مثلًا: تنفيذ جميع العمليات في الواجهة الأمامية قبل الخلفية).
 - قد يؤدي ذلك إلى مشكلة المجاعة (Starvation) حيث لا تُنفذ العمليات ذات الأولوية المنخفضة بسبب ازدحام القوائم الأعلى.
- تقسيم الوقت (Time Slice)
 - يُخصص وقت محدد لوحدة المعالجة لكل قائمة بناءً على نسبة مئوية.
 - 80% من زمن وحدة المعالجة مخصص للواجهة الأمامية باستخدام خوارزمية RR
 - 20% مخصص للواجهة الخلفية باستخدام خوارزمية FCFS

الموضوع العاشر | الجدولة متعددة المستويات (Multilevel Queue)

أعلى أولوية



أقل أولوية

- يوضح هذا الرسم كيفية تصنيف أنواع العمليات المختلفة في النظام إلى طبقات (قوائم متعددة) حسب مستوى الأولوية، بحيث يتم تنفيذ العمليات ذات الأولوية الأعلى أولاً، ثم الأقل.
- أعلى أولوية: تعني أن هذه العمليات يتم تنفيذها أولاً دائماً إذا كانت موجودة في قائمة الانتظار.
- أقل أولوية: لا يتم تنفيذها إلا إذا لم توجد عمليات ذات أولوية أعلى في القائمة.



الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

شكراً لكم



الكلية التطبيقية
الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432



دبلوم امن المعلومات

اسم المقرر: : مفاهيم انظمة التشغيل

رمز المقرر: OSC

الموضوع: التاسع



الموضوع التاسع جدولة وحدة المعالجة المركزية

الأهداف

في نهاية هذا الدرس، يتوقع من المتعلم أن يكون قادرًا على أن:

- تتعرف على مفهوم جدولة وحدة المعالجة المركزية (CPU) .
- تتعرف على مفهوم الموزع Dispatcher.
- تعدد مهام الموزع Dispatcher .
- تتعرف على معايير الجدولة .
- تتعرف على معايير تحسين خوارزميات الجدولة .
- تفرّق بين أنواع خوارزميات الجدولة .



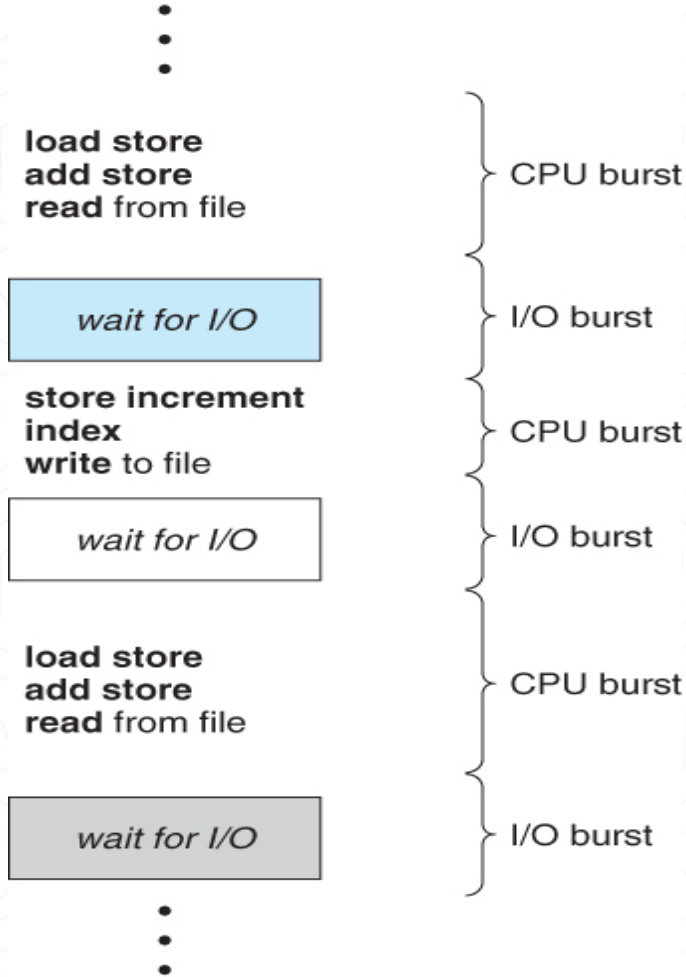
وصف الدرس

في هذا الدرس سنتناول:

- جدول وحدة المعالجة المركزية (CPU) .
- الرسم البياني للتوزيع التكراري لزمان استخدام وحدة المعالجة .
- الموزع Dispatcher .
- معايير الجدولة .
- معايير تحسين خوارزميات الجدولة.
- أنواع خوارزميات الجدولة .

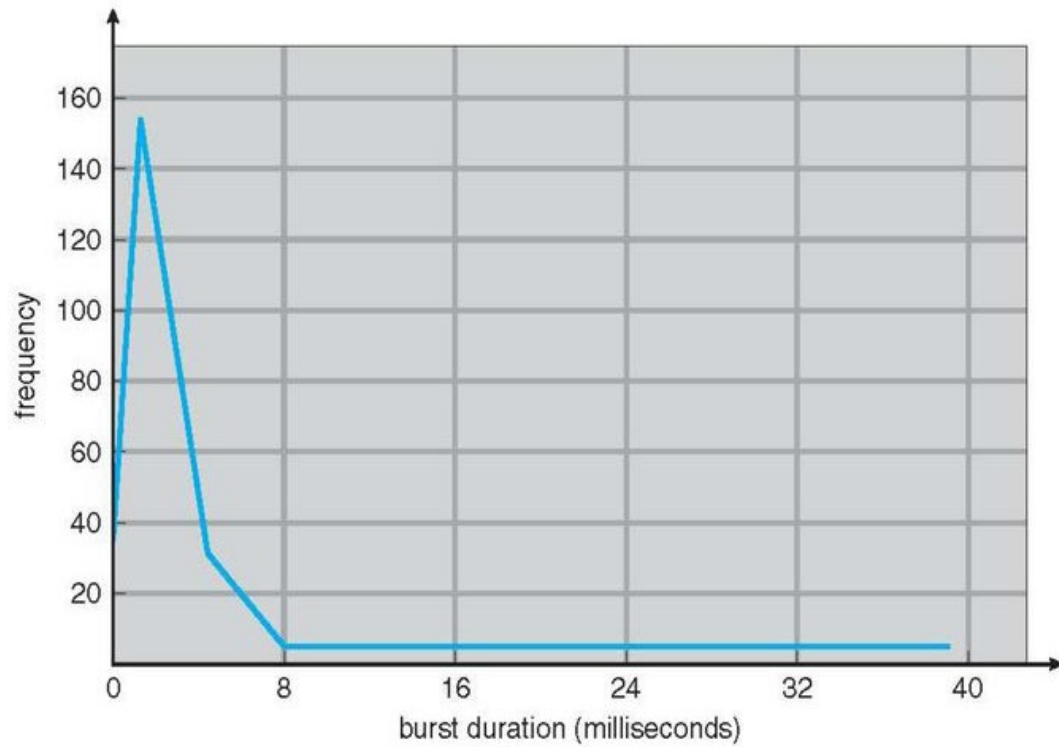


الموضوع التاسع | المفاهيم الأساسية



- تحقيق أقصى استفادة من وحدة المعالجة المركزية يتم عبر أسلوب البرمجة المتعددة.
- دورة وحدة المعالجة المركزية (CPU burst) هي الفترة الزمنية التي تستخدم فيها العملية وحدة المعالجة لتنفيذ التعليمات.
- دورة الإدخال/الإخراج (I/O burst) هي الفترة الزمنية التي تقضيها العملية في انتظار تنفيذ عمليات الإدخال أو الإخراج، مثل القراءة من الملفات أو الكتابة عليها.
- تتناوب العملية بين دورات CPU و I/O عدة مرات خلال فترة حياتها.
- تبدأ الدورة عادةً بتنفيذ التعليمات باستخدام وحدة المعالجة المركزية (CPU burst)، ثم تليها فترة انتظار لإجراء عمليات الإدخال/الإخراج (I/O burst).

الموضوع التاسع | الرسم البياني للتوزيع التكراري لزمان استخدام وحدة المعالجة المركزية



يوضح الرسم البياني توزيع مدد تنفيذ المهام باستخدام وحدة المعالجة المركزية (CPU burst durations) عبر مجموعة من العمليات. كما هو موضح، فإن الغالبية العظمى من المهام تتطلب مددًا زمنيًا قصيرة جدًا للتنفيذ، بينما تقل احتمالية وجود مهام طويلة المدة بشكل كبير. يشير هذا التوزيع إلى أن معظم العمليات تنتهي بسرعة، مما يُعزز كفاءة استخدام المعالج ويؤثر في استراتيجيات جدولة العمليات (scheduling).

الموضوع التاسع | جدول وحدة المعالجة المركزية

جدول وحدة المعالجة المركزية

- يُنفذ المجدول قصير الأمد مهمة اختيار عملية جاهزة من قائمة الانتظار لتخصيص وحدة المعالجة المركزية لها.
- يتم ترتيب قائمة العمليات الجاهزة وفقًا لمعايير محددة (مثل أولوية العملية أو زمن وصولها).
- يتم اتخاذ قرارات الجدولة في الحالات التالية:
 1. عند انتقال العملية من حالة التنفيذ إلى حالة الانتظار.
 2. عند انتقال العملية من حالة التنفيذ إلى حالة الجاهزية.
 3. عند انتقال العملية من حالة الانتظار إلى حالة الجاهزية.
 4. عند انتهاء العملية.
- الجدولة في الحالتين الأولى والرابعة تكون غير استباقية.
- جميع حالات الجدولة الأخرى تكون استباقية.

الموضوع التاسع | جدول وحدة المعالجة المركزية

• أنواع الجدولة:

• جدول غير استباقية:

- هي نوع من جدولة المعالج حيث لا يمكن مقاطعة العملية التي تعمل حاليًا على وحدة المعالجة المركزية. تستمر العملية في التنفيذ حتى تنتهي من عملها أو تنتقل إلى حالة الانتظار (مثل انتظار إدخال/إخراج). بمعنى آخر، لا يتم تخصيص المعالج لعملية أخرى إلا بعد أن تُنهي العملية الحالية استخدامه طواعية.

• جدول استباقية:

- هي نوع من جدولة المعالج حيث يمكن مقاطعة العملية الحالية التي تعمل على وحدة المعالجة المركزية. يحدث ذلك عندما تدخل عملية جديدة إلى قائمة الجاهزية وتمتلك أولوية أعلى، أو حسب سياسة النظام. يتم إيقاف العملية الحالية مؤقتًا، وتُعاد إلى قائمة الجاهزية لإتاحة المعالج للعملية ذات الأولوية.

الموضوع التاسع | جدولة وحدة المعالجة المركزية

لإدارة جدولة وحدة المعالجة المركزية وتجنب التعارضات في النظام هناك عدة عوامل يجب مراعاتها لضمان الأداء الأمثل للنظام:

- ضرورة التأكد من تنظيم الوصول إلى البيانات المشتركة بطريقة آمنة وفعالة.
- مراعاة خاصية الاستباقية عند تنفيذ المهام داخل النواة، وخاصة في الأنظمة التي تتطلب استجابة فورية.
- الاستعداد للتعامل مع المقاطعات المحتملة أثناء تنفيذ العمليات الحرجة أو عالية الأولوية، لتفادي أي تعارض في موارد النظام.

الموضوع التاسع | الموزع Dispatcher

الموزع

- الموزّع هو المكوّن الذي يتولّى التحكم في تسليم وحدة المعالجة المركزية للعملية التي اختارها المجدول قصير الأمد (short-term scheduler)
- مهام الموزع تشمل ما يلي:
 - تبديل السياق (Context Switching) حفظ حالة العملية الحالية وتحميل حالة العملية الجديدة.
 - التبديل إلى وضع المستخدم (User Mode) لضمان تنفيذ البرامج في بيئة محدودة الصلاحيات.
 - الانتقال إلى موقع التعليمات: استئناف التنفيذ من حيث توقفت العملية السابقة.
 - زمن انتقال التوزيع:
- هو الوقت المستغرق من لحظة إيقاف عملية إلى تشغيل عملية أخرى، ويؤثر بشكل مباشر على أداء النظام واستجابته

الموضوع التاسع | معايير الجدولة

معايير الجدولة

- استخدام وحدة المعالجة المركزية: الحرص على إبقاء وحدة المعالجة المركزية في حالة تشغيل مستمر لتحقيق أقصى استفادة ممكنة من موارد النظام.
- معدل الإنتاجية: قياس عدد العمليات التي يتم تنفيذها وإنجازها خلال فترة زمنية محددة.
- وقت التنفيذ: الزمن الإجمالي الذي تستغرقه العملية منذ تقديمها للنظام وحتى الانتهاء منها بشكل كامل
- وقت الانتظار: مجموع الفترات الزمنية التي تبقى فيها العملية في قائمة الجاهزية دون أن تُمنح وقتًا للتنفيذ.
- وقت الاستجابة: الفترة الزمنية بين إرسال الطلب إلى النظام وتلقّي أول استجابة تدل على بدء المعالجة، دون انتظار اكتمال التنفيذ.

الموضوع التاسع | معايير تحسين خوارزميات الجدولة

معايير تحسين خوارزميات الجدولة

- أقصى استخدام لوحدة المعالجة المركزية.
- رفع معدل الإنتاجية من خلال تنفيذ أكبر عدد ممكن من العمليات في أقل وقت.
- تقليل وقت التنفيذ لكل عملية إلى الحد الأدنى
- تقليل وقت الانتظار للعمليات قبل البدء في التنفيذ.
- تقليل زمن الاستجابة من لحظة تقديم الطلب وحتى أول استجابة من النظام

الموضوع التاسع | أنواع خوارزميات الجدولة

أنواع خوارزميات الجدولة

1. جدولة القادم أولاً، يُخدم أولاً (FCFS).
2. جدولة اقصر زمن تشغيل أولاً (SJN).
3. جدولة أقصر وقت متبقي ((Shortest Remaining Time).
4. جدولة الأولوية (Priority Scheduling).
5. جدولة التقسيم الزمني (Round Robin - RR).
6. جدولة متعددة المستويات (Multiple-Level Queues Scheduling).

جدولة القادم أولاً، يُخدم أولاً (FCFS):

- تقوم هذه الخوارزمية بجدولة العمليات حسب ترتيب وصولها إلى النظام. العملية التي تصل أولاً يتم تنفيذها أولاً، دون النظر إلى زمن تنفيذها.

الموضوع التاسع

جدولة القادم أولاً، يُخدم أولاً First- Come, First-Served (FCFS) Scheduling

<u>Process</u>	<u>(Burst Time)</u>
P ₁	24
P ₂	3
P ₃	3

• افترض أن العمليات تصل بالترتيب: P₃ ، P₂ ، P₁

• مخطط جانـت (Gantt Chart) للجدولة هو:



• زمن الانتظار:

- $P_1 = 0 ; P_2 = 24 ; P_3 = 27$

• متوسط زمن الانتظار:

- $(27+24+ 0) / 3 = 17$

الموضوع التاسع

جدولة القادم أولاً، يُخدم أولاً First- Come, First-Served (FCFS) Scheduling

- افتراض أن العمليات تصل بالترتيب: $P_2 ; P_3 ; P_1$

- مخطط جانـت (Gantt Chart) للجدولة هو:



- زمن الانتظار لـ $P_1 = 6$, $P_2 = 0$, $P_3 = 3$

- متوسط زمن الانتظار: $3 = (3+0+6) / 3$

- أفضل من الحالة السابقة.

- تأثير القافلة (Convoy effect) يُشير إلى الحالة التي تأخر العمليات القصيرة بسبب انتظارها لإنهاء العمليات الطويلة.
 - حالة يجب مراعاتها:
- عندما تكون هناك عملية تحتاج لوحدة المعالجة المركزية (CPU) وأخرى تعتمد على الإدخال/الإخراج قد تتسبب عمليات الإدخال/الإخراج في تأخير العملية التي تحتاج لا CPU. لذلك، يجب إدارة الجدولة بشكل عادل بين العمليات.

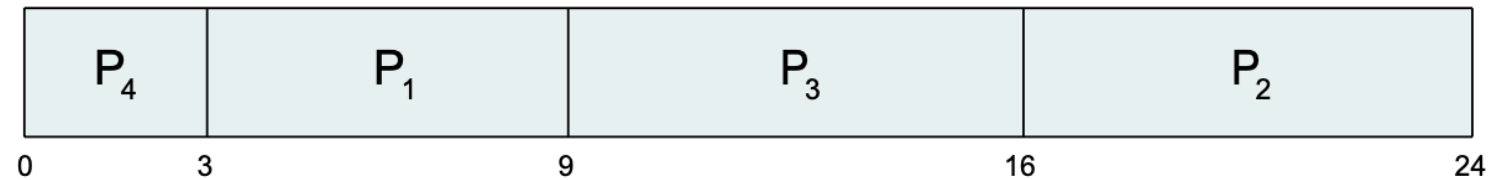
جدولة أقصر زمن تشغيل أولاً Shortest-Job-First Scheduling (SJF)

- يتم ربط كل عملية بطول زمن التشغيل القادم لوحدة المعالجة المركزية (CPU Burst).
- تُستخدم هذه الأطوال لجدولة العملية ذات الزمن الأقصر.
- خوارزمية جدولة أقصر زمن تشغيل أولاً SJF مثالية، حيث تحقق أقل متوسط زمن انتظار لمجموعة معينة من العمليات.
- التحدي هو معرفة طول طلب وحدة المعالجة المركزية القادم، ويمكن طلب هذه المعلومة من المستخدم.

مثال - جدولة أقصر زمن تشغيل أولاً SJF

<u>Process</u>	<u>(Burst Time)</u>
P ₁	6
P ₂	8
P ₃	7
P ₄	3

- مخطط الجدولة لخوارزمية أقصر زمن تشغيل أولاً (SJF Scheduling Chart)



- متوسط زمن الانتظار = $7 = (0 + 9 + 16 + 3)/4$

الموضوع التاسع | مثال - جدولة أقصر زمن تشغيل أولاً SJF

تناولنا في هذا الدرس مفهوم جدولة وحدة المعالجة المركزية ودورها في تحسين استغلال المعالج عبر اختيار العملية المناسبة من قائمة الجاهزية. كما تم شرح دورات المعالج (CPU Burst) ودورات الإدخال/الإخراج (I/O Burst) وتأثيرها على سلوك العمليات. وتطرقنا إلى أنواع الجدولة الاستباقية وغير الاستباقية، ودور الموزع (Dispatcher) ومهامه مثل تبديل السياق وزمن الانتقال. كما تم توضيح معايير الجدولة ومعايير تحسين خوارزمياتها مثل تقليل وقت الانتظار وزمن الاستجابة وزيادة الإنتاجية. واختتمنا الدرس بعرض أنواع خوارزميات الجدولة المختلفة مثل FCFS و SJF و Round Robin وتأثير القافلة على أداء النظام.



الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

شكراً لكم



الكلية التطبيقية
الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432



مفاهيم أنظمة التشغيل :

OSC :

:



مزامنة العمليات

في نهاية هذا الدرس، يتوقع من المتعلم أن يكون قادرًا على أن:

- يشرح مفهوم تزامن العمليات في أنظمة التشغيل.
- يوضح مشكلة القسم الحرج وأبعادها.
- يفسر خوارزمية العملية P_i وآلية عملها.
- يحلل إدارة القسم الحرج في نظام التشغيل.
- يطبق تعليمات الاختبار والتعيين في معالجة التزامن.
- يميز بين تعليمات المقارنة والمبادلة ودورها في إدارة العمليات.
- يستعرض أمثلة تطبيقية على آليات التزامن.





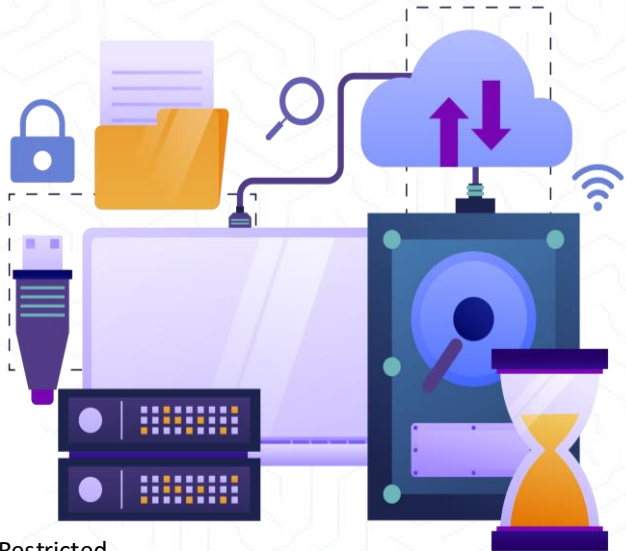
في هذا الدرس سنتناول:

- مزامنة العمليات ومشكلة المنتج والمستهلك.
- حالة التنافس ومشكلة القسم الحرج.
- الخوارزمية للعملية P_i وحلول القسم الحرج.
- حل بيترسون كأحد نماذج التزامن.
- أجهزة التزامن ودورها في إدارة العمليات.
- تعليمات الاختبار والتعيين وتعليمات المقارنة والمبادلة

الموضوع الثامن | مزامنة العمليات Process Synchronization

مقدمة

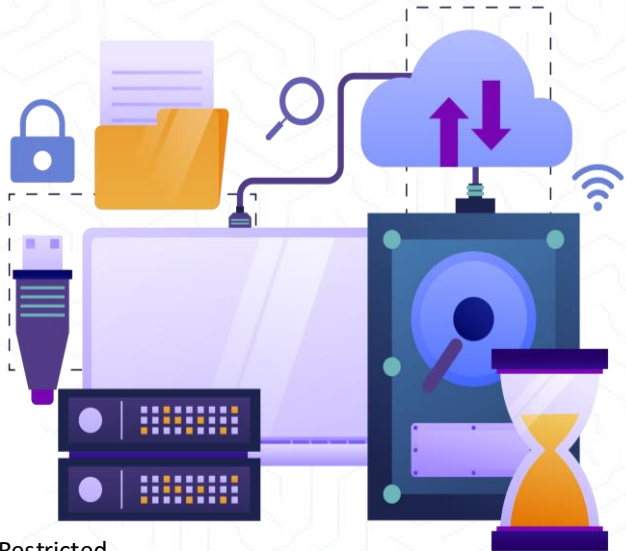
- يمكن أن تنفذ العمليات بشكل متوازي. وقد يتم إيقاف عملية واستئنافها في أي وقت
- مشاركة البيانات بين العمليات المتزامنة قد يؤدي إلى تضارب وفقدان الاتساق.
- ضمان الوصول المنظم للبيانات المشتركة يتطلب آليات تحكم دقيقة داخل نظام التشغيل.
- الحفاظ على الاتساق يتطلب استخدام آليات مزامنة لضمان تنفيذ العمليات المتعاونة بشكل صحيح.



الموضوع الثامن | مزامنة العمليات Process Synchronization

توضيح المشكلة:
نُفترض وجود عمليتين:

- المنتج (Producer) الذي يُنتج عناصر ويضعها في المخزن المؤقت.
- المستهلك (Consumer) الذي يسحب هذه العناصر من المخزن المؤقت ويستهلكها.
- يُستخدم عداد (Counter) لمتابعة عدد العناصر داخل المخزن المؤقت.
- عند إضافة عنصر جديد بواسطة المنتج، يتم زيادة العداد.
- عند استهلاك عنصر من قبل المستهلك، يتم إنقاص العداد.
- في حالة عدم وجود مزامنة:
- قد يحدث تضارب إذا حاول المنتج والمستهلك تعديل العداد في نفس الوقت.
- يؤدي ذلك إلى نتائج غير متوقعة أو تلف في البيانات.



المنتج Producer

الموضوع الثامن

```
while (true) {  
    /* produce an item in next_produced */  
    while (counter == BUFFER_SIZE)  
        // do nothing */  
  
    buffer[in] = next_produced;  
    // Place the produced item  
    // in the buffer  
  
    in = (in + 1) % BUFFER_SIZE;  
    // Update the index to the next  
    // position in the buffer  
  
    counter++;  
    // Increment the counter to reflect  
    // a new item in the buffer  
}
```

في نظام التشغيل، يمثل المنتج (Producer) جزءًا من مشكلة "المنتج والمستهلك"، حيث يقوم بإنشاء عناصر جديدة (بيانات أو مهام) ويضعها في مخزن مشترك (Buffer) لاستخدامها لاحقًا من قبل المستهلك.

في هذا الكود، يعمل المنتج بشكل مستمر داخل حلقة لا نهائية. إذا كان المخزن ممتلئًا (أي أن العداد Counter يساوي حجم المخزن BUFFER_SIZE يتوقف المنتج مؤقتًا عن الإضافة.

بمجرد توفر مساحة، يُنتج عنصر جديد ويوضع في الموضع الحالي للمؤشر in، ثم يُحدث المؤشر ليشير إلى الموقع التالي باستخدام العملية الدائرية (الـ modulo). بعد ذلك، يُزاد العداد للإشارة إلى وجود عنصر جديد في المخزن.

هذا النموذج يُظهر كيفية إدارة الوصول إلى الموارد المشتركة بدون تزامن، مما يهيئ لفهم أهمية آليات المزامنة لاحقًا مثل mutex أو semaphores.

```
while (true) {  
    while (counter == 0)  
        // do nothing */  
  
    next_consumed = buffer[out];  
    // Retrieve the item from  
    // the buffer to consume it  
  
    out = (out + 1) % BUFFER_SIZE;  
    // Move the output index to  
    // the next position  
  
    counter--;  
    // Decrease the counter to reflect  
    // consumption of an item  
}
```

يظهر في هذا الجزء من الشيفرة الدور الذي يقوم به المستهلك في نظام منتج-مستهلك. تبدأ العملية بحلقة لا نهائية، حيث ينتظر المستهلك حتى يتوفر عنصر داخل المخزن المؤقت buffer

إذا كان العداد Counter يساوي صفرًا، فهذا يعني أن المخزن فارغ، ويستمر المستهلك في الانتظار. بمجرد توفر عنصر، يتم استخراجه من الموقع المشار إليه بـ out ثم يتم تحديث موقع المؤشر ليشير إلى الموضع التالي باستخدام معامل باقي القسمة الدائرية لضمان الالتفاف داخل المخزن، وأخيرًا يُنقص العداد للإشارة إلى استهلاك عنصر.

هذه الآلية تضمن استهلاك البيانات بالتزامن مع الإنتاج، دون فقدان أو تكرار للبيانات.

حالة السباق (Race Condition) :

هي مشكلة تحدث في الأنظمة متعددة العمليات أو الخيوط عندما تحاول عدة عمليات الوصول إلى نفس المورد المشترك وتعديله في نفس الوقت دون وجود آلية تزامن مناسبة، مما يؤدي إلى نتائج غير صحيحة أو غير متوقعة بسبب اختلاف ترتيب تنفيذ العمليات.

- كيف تحدث حالة السباق (Race Condition) ؟
- تحدث المشكلة عندما يكون هناك عداد مشترك بين عمليتين كما يلي:
- العملية (أ) تقرأ قيمة العداد ثم تزيده بمقدار +1.
- العملية (ب) تقرأ نفس قيمة العداد ثم تزيده بمقدار +1 أيضًا.
- إذا تم تنفيذ العمليتين في نفس الوقت، فقد تحصلان على نفس القيمة القديمة قبل أن يتم تحديثها.
- هذا يؤدي إلى فقدان إحدى عمليات الزيادة، مما ينتج عنه قيمة غير صحيحة للعداد.
- السبب في ذلك هو أن العمليتين تتسابقان لتعديل نفس المورد المشترك دون تزامن مناسب.
- كيف نتجنب حالة السباق؟
- يكون تجنب ذلك من خلال التعامل مع القسم الحرج كقسم لا يمكن الوصول إليه إلا بعملية واحدة في كل مرة. يسمى هذا النوع من الأقسام القسم الذري.
- أي غير القابل للتجزئة.

- القسم الحرج:
- هو الجزء من الشيفرة الذي يتم فيه الوصول إلى الموارد المشتركة مثل المتغيرات أو الملفات، ويجب التحكم في الوصول إليه لمنع التداخل.
- مشكلة القسم الحرج:
- هي التحدي في تصميم آلية تزامن تضمن أن عملية واحدة فقط يمكنها الدخول إلى القسم الحرج في كل مرة، لمنع حدوث تداخل في البيانات أو أخطاء التزامن مثل حالة السباق.

مشكلة القسم الحرج

- افترض وجود نظام يحتوي على n عملية $\{p_0, p_1, \dots, p_{n-1}\}$
- كل عملية تحتوي على جزء من الشيفرة يُسمى "القسم الحرج".
- قد تقوم العمليات بتغيير المتغيرات المشتركة، تحديث الجداول، كتابة الملفات، إلخ.
- عندما تكون إحدى العمليات في القسم الحرج، لا يُسمح لأي عملية أخرى بدخول القسم الحرج في نفس الوقت.
- تكمن مشكلة القسم الحرج في تصميم بروتوكول لحل هذه المشكلة.
- يجب على كل عملية طلب الإذن للدخول إلى القسم الحرج في قسم الدخول.
- يمكن أن يتبع القسم الحرج قسم الخروج، ثم قسم المتبقي.

- أقسام البرنامج في حل مشكلة التزامن:

do {

entry section

critical section

exit section

remainder section

} while (true);

- قسم الدخول (Section Entry): يقرر قسم الدخول دخول العملية.
- القسم الحرج (Section Critical): يسمح القسم الحرج ويتأكد من أن عملية واحدة فقط تعدل البيانات المشتركة.
- قسم الخروج (Section Exit): يتم التعامل مع إدخال العمليات الأخرى في البيانات المشتركة بعد تنفيذ عملية واحدة بواسطة قسم الخروج.
- القسم المتبقي (Section Remainder): الجزء المتبقي من الكود الذي لم يتم تصنيفه على النحو الوارد أعلاه موجود في القسم المتبقي.

- الحل المطلوب هو تصميم بروتوكول تزامن (Synchronization Protocol) يمنع حدوث مشكلة القسم الحرج بحيث:
 - لا يُسمح لأكثر من عملية واحدة بالدخول إلى القسم الحرج في نفس الوقت (التداخل المتبادل).
 - أي عملية ترغب في الدخول إلى القسم الحرج يجب أن تطلب الإذن أولاً عبر قسم الدخول.
 - بعد إنهاء عملها، يجب أن تخرج وتحرر الموارد عبر قسم الخروج.
 - الجزء المتبقي من الكود، الذي لا يؤثر على البيانات المشتركة، يُسمى القسم المتبقي.
- يتم التحكم في دخول العمليات إلى القسم الحرج باستخدام تعليمات مثل:
 - wait(): تمنع العمليات الأخرى من الدخول إلى القسم الحرج حتى تنتهي العملية الحالية.
 - signal(): تسمح بخروج العملية الحالية، مما يتيح لعملية أخرى الدخول إلى القسم الحرج.



- استبعاد التزامن - إذا كانت العملية P_i تنفذ في قسمها الحرج، فلا يمكن لأي عملية أخرى أن تنفذ في أقسامها الحرجة.
- التقدم - إذا لم تكن أي عملية تنفذ في قسمها الحرج وكان هناك بعض العمليات التي ترغب في دخول قسمها الحرج، فإن اختيار العمليات التي ستدخل القسم الحرج بعد ذلك لا يمكن تأجيله إلى أجل غير مسمى.
- الانتظار المحدود - يجب أن يكون هناك حد لعدد المرات التي يُسمح فيها للعمليات الأخرى بالدخول إلى أقسامها الحرجة بعد أن تطلب عملية الدخول إلى قسمها الحرج وقبل أن يُسمح لها بالدخول.
- افترض أن كل عملية تنفذ بسرعة غير صفرية
- لا يوجد افتراض بشأن السرعة النسبية للعمليات

إدارة القسم الحرج في نظام التشغيل

- تهدف إلى التحكم في الوصول إلى الموارد المشتركة بين العمليات لتجنب التداخل وحالات السباق.
- هناك طريقتان يعتمد اختيار إحدهما على ما إذا كان نواة النظام قابلة للإيقاف التلقائي أو غير قابلة للإيقاف التلقائي:
- قابلة للإيقاف التلقائي (Preemptive) - يسمح بالإيقاف التلقائي للعمليات عندما تكون قيد التنفيذ في وضع النواة.
- غير قابلة للإيقاف التلقائي (Non-preemptive) - تعمل العملية حتى تخرج من وضع النواة أو تقوم بحظر نفسها أو تمنح المعالج طوعاً.
- خالية أساساً من حالات التنافس في وضع النواة.

حل بيترسون هو خوارزمية لحل مشكلة القسم الحرج باستخدام عمليتين فقط.

- يعتمد حل بيترسون على فكرة أنه عندما تنفذ عملية في القسم الحرج، فإن العملية الأخرى تقوم بتنفيذ باقي التعليمات البرمجية الخاصة بها خارج القسم الحرج، والعكس صحيح. أي لا يمكن مقاطعتها أثناء تنفيذها.
 - في حل بيترسون، هناك متغيرين مشتركين تستخدمهما العمليات للتحكم في الوصول إلى القسم الحرج:
 - `turn`: يحدد أي عملية يحق لها الدخول إلى القسم الحرج في الوقت الحالي.
 - `flag[i]`: مصفوفة تحتوي على عنصرين:
1. إذا كانت `flag[i] = true` فهذا يعني أن العملية `Pi` جاهزة لدخول القسم الحرج.
 2. إذا كانت `flag[i] = false` فهذا يعني أن العملية `Pi` ليست جاهزة لدخول القسم الحرج في تلك اللحظة.

حل بيترسون

- يمكن إثبات أن الثلاثة متطلبات للقسم الحرج تم استيفائها:

1. الاستبعاد المتبادل محفوظ:

- تدخل العملية P_i في القسم الحرج فقط إذا:

- إما أن $flag[j] = false$

أو

- $turn = i$

2. متطلب التقدم تم استيفائها.

3. متطلب الانتظار المحدود تم استيفائها.

أجهزة التزامن:

- أجهزة التزامن تساعد في حل مشكلة القسم الحرج. باستخدام الأقفال.
- في الأنظمة الأحادية المعالج، يمكن تعطيل المقاطعات لضمان عدم التداخل أثناء تنفيذ الكود.
- هذا الحل غير فعال في الأنظمة متعددة المعالجات، حيث لا يمكن تعطيل المقاطعات بسهولة لأن كل معالج يعمل بشكل مستقل ولا يمكنه منع المقاطعات في المعالجات الأخرى.
- في الأجهزة الحديثة، تستخدم تعليمات ذرية (غير قابلة للمقاطعة) لضمان تنفيذ العمليات في الأقسام الحرجة بأمان.
- مثل: اختبار وتعيين قيمة الذاكرة.
- تبادل محتويات كلمتين في الذاكرة.

مثال عملي:

استخدام Test-and-Set لإنشاء قفل (Lock):

- ◆ إذا كان $lock = 0$ ، فهذا يعني أن القسم الحرج متاح للدخول.
- ◆ إذا كان $lock = 1$ ، فهذا يعني أن القسم الحرج مشغول.

أقفال الاستبعاد المتبادل (Mutex)

- قفل الـ Mutex هو حل بسيط لمشكلة القسم الحرج.
- يتم حماية القسم الحرج باستخدام عمليتين: `acquire()` لحجز القفل و `release()` لإطلاق القفل.
- متغير `boolean` يحدد ما إذا كان القفل متاحًا أو لا.
- يجب أن تكون العمليتين ذرية (لا يمكن مقاطعتها).
- عادةً ما يتم تنفيذ هذه العمليات باستخدام تعليمات ذرية في الأجهزة.
- هذا الحل يعتمد على انتظار مشغول (`busy waiting`) حيث تبقى العملية في حالة انتظار حتى يتم تحرير القفل.
- يُسمى هذا النوع الأقفال الدوارة (`Spinlock`).

الموضوع الثامن | السيمافور (Semaphore)

السيمافور هو أداة تستخدم للتحكم في الوصول إلى الموارد المشتركة بين العمليات بشكل منظم.

- هو متغير صحيح يمكن أن يأخذ قيمًا غير سالبة.

- يتم استخدام عمليتين للتحكم فيه:

 - . wait()

 - . signal()

- السيمافور يمكن أن يكون ثنائيًا (بقيمة 0 أو 1، يشبه القفل) أو عدديًا (يتم تحديده باستخدام قيم أكبر من 1).

- الهدف هو تنظيم التزامن بين العمليات ومنع حدوث التداخل أو الوصول المتزامن للموارد.

الموضوع الثامن | الجوع والجمود (Deadlock and Starvation)

الجوع والجمود

- الجمود يحدث عندما تنتظر عمليتان أو أكثر بشكل غير محدد حدوث حدث لا يمكن أن يحدث إلا بواسطة واحدة من العمليات المنتظرة.
- افترض وجود سيمافورين S و Q حيث يتم تهيئتهما بقيمة ابتدائية تساوي 1.

P0	P1
<code>wait(S) ;</code>	<code>wait(Q) ;</code>
<code>wait(Q) ;</code>	<code>wait(S) ;</code>
<code>...</code>	<code>...</code>
<code>signal(S) ;</code>	<code>signal(Q) ;</code>
<code>signal(Q) ;</code>	<code>signal(S) ;</code>

- الجوع: حجب غير محدود
- حيث قد لا تتم إزالة العملية من قائمة انتظار السيمافور التي تم تعليقها فيها.
- انعكاس الأولوية:
- تحدث هذه المشكلة عندما تحتفظ عملية ذات أولوية منخفضة بقلل تحتاجه عملية ذات أولوية أعلى.
- يتم حل المشكلة باستخدام بروتوكول وراثّة الأولوية.

الموضوع الثامن | أمثلة تزامن (تزامن سولاريس)

تزامن سولاريس

- يُنفذ مجموعة من الأقفال لدعم تعدد المهام، والخيوط المتعددة (بما في ذلك الخيوط الزمنية الحقيقية)، والمعالجة المتعددة.
- يستخدم الأقفال المتكيفة لتحقيق الكفاءة عند حماية البيانات من المقاطع البرمجية القصيرة.
- يبدأ كقفل دوار بواسطة الإشارة القياسية.
- إذا كان القفل محجوزًا، وكان بواسطة خيط يعمل على وحدة معالجة مركزية أخرى، فإنه يدور.
- إذا كان القفل محجوزًا بواسطة خيط غير في حالة تشغيل، فإنه يتم حظره والنوم في انتظار إشارة لإطلاق القفل.
- يستخدم متغيرات الحالة.
- يستخدم أقفال القارئ-الكاتب عندما تحتاج أجزاء أطول من الكود إلى الوصول إلى البيانات.
- يستخدم البوابات الدوارة لترتيب قائمة الخيوط التي تنتظر للحصول على قفل متكيف أو قفل قارئ-كاتب.
- البوابات الدوارة تكون لكل خيط حامل للقفل، وليس لكل كائن
- تُعطي وراثية الأولوية لكل بوابة دوارة أعلى الأولويات بين الخيوط في بوابتها الدوارة.